

Master Thesis

Title

by

Mihir Mahesh Ambli

7th October, 2026

University of Kaiserslautern-Landau
Department of Computer Science
Distributed Systems Lab

Examiner: Prof. Dr.-Ing. Jens B. Schmitt
M. Sc. Daniel Reti

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die von mir vorgelegte Arbeit mit dem Thema „Title “ selbstständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel (auch generative KI-Systeme) vollständig angegeben habe und dass ich die Stellen der Arbeit — einschließlich Tabellen und Abbildungen —, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind unter Angabe der Quelle als Entlehnung kenntlich gemacht habe. Sofern der Einsatz generativer KI-Systeme über eine reine Grammatikprüfung und Stilverbesserung hinausgeht, ist der Einsatz nachvollziehbar kenntlich gemacht. Für Fehler im Text oder Code bin ich verantwortlich, auch bei Übernahme aus generativen KI-Systemen. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Kaiserslautern, den 7.10.2026

Mihir Mahesh Ambli

Abstract

English

Zusammenfassung

Deutsche

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Questions . . . 3
- 1.4 Thesis Outline . . . 3

2 Background **4**
- 2.1 Cyber Deception, Honeypots and Honeytokens . . . 4
- 2.2 Web Application Reconnaissance . . . 4
- 2.3 Reverse Proxies and Their Request Phases . . . 4

3 Related Work **5**

4 Design of the Deception Layer **6**
- 4.1 Requirements and Threat Model . . . 6
- 4.2 Architecture . . . 6
- 4.3 Honeytoken Kinds . . . 6
  - 4.3.1 HTML Comments . . . 6
  - 4.3.2 HTTP Headers . . . 6
  - 4.3.3 Cookies . . . 6
  - 4.3.4 Decoy Paths . . . 7
  - 4.3.5 Form Fields . . . 7
- 4.4 The Fake SQL-Injection Trap . . . 7
- 4.5 Detection and Response Pipeline . . . 7

5 Implementation on Four Deception layer platforms **8**
- 5.1 Common Structure . . . 8
- 5.2 OpenResty (Reference Edge) . . . 8
- 5.3 Envoy with a Lua Filter . . . 8
- 5.4 Envoy with a WebAssembly Filter . . . 8
- 5.5 Apache httpd with mod_lua . . . 9
- 5.6 Levelling the Edges . . . 9
- 5.7 Verifying Equivalence . . . 9

6 Evaluation Methodology **10**
- 6.1 Goals and Metrics . . . 10
- 6.2 Testbed . . . 10

6.3 Workload . . . 10
6.4 Measurement Planes . . . 10
6.5 Run Validity . . . 11

7 Results **12**
7.1 RQ1: Latency Added . . . 12
7.1.1 Median Latency per GET across Rates . . . 12
7.1.2 Tail Latency . . . 12
7.1.3 Request Position and Reconnects . . . 12
7.1.4 Answer to RQ1 . . . 12
7.2 RQ2: CPU and Capacity . . . 12
7.2.1 CPU per Request . . . 12
7.2.2 Cost Attribution . . . 13
7.2.3 The Cost of Measuring . . . 13
7.2.4 Per-Mechanism Cost and Page Size . . . 13
7.2.5 Answer to RQ2 . . . 13
7.3 RQ3: Functional Correctness . . . 13
7.3.1 Response Integrity under Load . . . 13
7.3.2 Detection Recall and False Positives . . . 13
7.3.3 Answer to RQ3 . . . 13
7.4 RQ4: Influence of the Extension Runtime . . . 14
7.4.1 Per-Kind Operation Cost . . . 14
7.4.2 The SQLi Trap: Scan Depth versus Decoding . . . 14
7.4.3 Stability of Apache mod_lua . . . 14
7.4.4 Answer to RQ4 . . . 14

8 Discussion **15**
8.1 Synthesis across Research Questions . . . 15
8.2 Choosing an Edge for Deception . . . 15
8.3 Where the Cost Comes From . . . 15
8.4 Deployment Implications . . . 15
8.5 Threats to Validity . . . 15
8.6 Limitations . . . 15

9 Conclusion and Future Work **16**
9.1 Summary . . . 16
9.2 Future Work . . . 16

A Configuration Reference **17**

B Parity-Check Request Set **18**

C Reproducing the Benchmarks **19**

D Additional Result Tables **20**

1 Introduction

A web application exposes more to its visitors than the pages it renders. Its HTML source, its response headers and cookies, and auxiliary files such as `robots.txt` are delivered to every client, including one that is preparing an attack. This thesis studies a defence that turns that exposure against the attacker: an edge proxy in front of the application plants honeytokens in the traffic it forwards and raises an alert when one of them is used. The mechanism developed for this purpose is called the Reverse-Proxy Deception Layer (RPDL). The thesis implements RPDL on four reverse-proxy platforms and measures what it costs on each.

1.1 Motivation

Security testing of a web application begins with information gathering. The OWASP Web Security Testing Guide instructs a tester to review the comments and metadata in a page's HTML source, which may reveal internal information that should not be available to an attacker, and to read `robots.txt` for paths and functionality that the site does not link to [1]. An adversary who reconnoitres the same application has access to the same artefacts, because they are served to any client that requests them.

Cyber deception exploits this phase of an attack [2]. Spitzner describes a honeypot as an information system resource whose value lies in unauthorised or illicit use of that resource [3]; a honeytoken applies the same idea to a piece of data, such as a fake credential or a URL parameter that no legitimate workflow uses [4]. Because a legitimate user has no reason to touch such an element, an interaction with it is a strong indication of malicious activity [2], and deception products have been marketed as intrusion detection with zero false positives [4]. Han et al. tested this claim for web applications [2]. Over seven months of production use with 258 regular users, their deceptive elements raised no false alarm. In a red-team exercise with 150 participants, 84 participants triggered at least one of twelve traps, and 64 % of the participants who found a real vulnerability also triggered a trap. The same study found that 36 % of the attackers who exploited at least one vulnerability did so without triggering any trap, and concluded that deception is a valuable companion to other detection techniques rather than a standalone protection [2].

The practical obstacle is deployment. Kahlhofer and Rass observe that what limits the adoption of cyber deception is its deployment in real-world software systems,

and that the operator responsible for security usually has access to the built software artefacts but not to their source code [5]. Of the 19 technical methods they review for deploying application-layer deception without developer involvement, placing a reverse proxy in front of the application is the one most commonly found in research [5]. Han et al. implemented their framework as a transparent reverse proxy that injects cookies, hidden form fields and `robots.txt` entries into responses and removes them from incoming requests before forwarding [2]. Cloxy couples a reverse proxy with several deception-based defences to protect legacy web applications that cannot easily be updated [6], and Honey Infiltrator injects honeytokens, such as an HTML comment, into web traffic from a layer-2 network bridge [7].

A reverse proxy is frequently present already. In Kubernetes, the ingress that connects a cluster to the outside world is often a reverse proxy such as NGINX or Envoy [5], and such proxies can be extended with custom code: Envoy, for example, accepts filters written in C++, Lua or WebAssembly, and developers of custom filters tend to use Lua or WebAssembly [8]. Placing the honeytokens in an edge proxy that the operator already runs therefore requires no change to the application and no additional network hop.

1.2 Problem Statement

Running deception logic inside the request path is not free. Kahlhofer and Rass note that the reverse-proxy pattern often comes with a significant performance overhead and call for research to reduce it [5]. The measurements they refer to come from service meshes, where Zhu et al. report up to 269 % higher latency and up to 163 % more virtual CPU cores for their benchmark applications, with the severity depending on configuration and workload [8]. The same study shows that the choice of extension runtime matters on its own: in Envoy, even a filter that does nothing has a substantial overhead when it is written in Lua or WebAssembly, and the Lua filter cost three times as much latency and nearly five times as much CPU as the WebAssembly filter [8]. A filter that does nothing, however, performs no deception. It bounds the cost of hosting an extension, not the cost of parsing requests for trigger values, rewriting response bodies and recording alerts.

The deception systems discussed above do not close this gap. Cloxy reports that its prototype increases the delay of communication with the protected web application, a delay its authors consider tolerable [6]. The evaluation by Han et al. concerns false alarms and detection accuracy [2]. Neither compares the cost of one deception mechanism across different proxy technologies, so the choice of edge cannot be made on measured grounds.

The problem addressed by this thesis is therefore the following. An operator who wants to deploy honeytokens at the web edge has no measured basis for choosing the

edge technology, nor for estimating what the deception costs in latency, CPU, capacity and functional correctness. The thesis answers this question for one deception mechanism on four reverse-proxy platforms in a controlled testbed. All measurements were taken on a single virtualised host serving a static origin, so the comparison between edges is the claim the results support, while absolute values are not expected to carry over to other hardware. The effectiveness of honeytokens against real attackers is outside the scope of this work; detection is tested only against scripted requests whose expected alerts are known.

1.3 Research Questions

The thesis addresses four research questions. Chapter 7 answers them in this order.

RQ1 How much latency does RPDL add to a request on each edge, and how does the added latency change with load?

RQ2 How much CPU and capacity does RPDL cost on each edge, and where does that cost arise?

RQ3 Does RPDL preserve the responses the edge produced without it, and does it raise the alerts it is expected to raise?

RQ4 To what extent do the answers to RQ1–RQ3 depend on the extension runtime: LuaJIT, PUC Lua or WebAssembly?

1.4 Thesis Outline

Chapter 2 introduces cyber deception and honeytokens, the reconnaissance that web honeytokens target, and the request-processing phases of a reverse proxy. Chapter 3 reviews prior work on honeytokens and decoys, application-layer deception without code changes, proxy and service-mesh overhead, and benchmarking methodology. Chapter 4 presents the design of RPDL independently of any edge, and Chapter 5 describes its implementation on the four reverse-proxy platforms together with the levelling that makes them comparable. Chapter 6 describes the testbed, the workload, the measurement planes and the criteria for a valid run. Chapter 7 presents the results, with one section per research question. Chapter 8 discusses the results, the choice of an edge for deception, the threats to validity and the limitations, and Chapter 9 concludes and outlines future work. The appendices contain the configuration reference, the parity-check request set, instructions for reproducing the benchmarks and additional result tables.

2 Background

The concepts a reader needs before the design makes sense.

2.1 Cyber Deception, Honeypots and Honeytokens

Spitzner's definition [9, 10]; honeytokens as data with no legitimate use; why false positives are rare.

2.2 Web Application Reconnaissance

What attackers read and tamper with: HTML source, response headers, cookies, `robots.txt`, hidden form fields, SQL-injection probes.

2.3 Reverse Proxies and Their Request Phases

Request hooks, header and body filters, body buffering, upstream connection pools.

3 Related Work

One continuous literature survey, grouped by theme in paragraphs rather than sections, ending with where this thesis sits. Themes in order: decoy documents [11], honeywords [12], and deceptive web artefacts such as HTML comments, `robots.txt` entries and invisible links [13]; application-layer deception without code changes, where Kahlhofer and Rass review 19 techniques, reverse proxies among them [5]; proxy and service-mesh overhead, including Lua versus WebAssembly filters in Envoy [8]; and benchmarking flaws in systems security [14, 15].

4 Design of the Deception Layer

The mechanism on a conceptual level, independent of any one edge.

4.1 Requirements and Threat Model

The operator controls the edge but not the application. The attacker is a reconnaissance or scanning client. Non-goals: blocking traffic, real authentication.

4.2 Architecture

Client → edge → static origin. One shared `config.json`, an in-memory attacker-IP store, and the alert log as RPDL's only output.

Material: `CONTEXT.md` (architecture diagram)

4.3 Honeytoken Kinds

4.3.1 HTML Comments

Injected before `</body>`; detected when the trigger keyword appears in the query.

4.3.2 HTTP Headers

A decoy response header; detected when its value is replayed in a later request.

4.3.3 Cookies

`Set-Cookie` bait; detected on tamper, since a browser replays the value unchanged.

4.3.4 Decoy Paths

Advertised as a hidden link or in `robots.txt`; any request to the path matches.

4.3.5 Form Fields

A hidden `<input>` before `</form>`; detected on tamper.

4.4 The Fake SQL-Injection Trap

The edge answers `POST /api/login` itself: a fake MySQL error that reflects the payload on a signature hit, a canned denial otherwise. Signatures are literal strings, not regular expressions.

4.5 Detection and Response Pipeline

Parse once, scan, strip the parameter so the origin never sees it, record the IP, then write one alert with CR/LF neutralised against log forgery.

5 Implementation on Four Deception layer platforms

How the design maps onto each proxy, and what made the four comparable.

5.1 Common Structure

A table mapping each edge's request hook, header filter and body filter, so the subsections below need only describe the differences.

5.2 OpenResty (Reference Edge)

Configuration compiled once in `init_by_lua`, detection in `access_by_lua`, injection in the header and body filters, attacker IPs in a `lua_shared_dict`.

Material: `nginx/README.md`, `nginx/nginx.conf`

5.3 Envoy with a Lua Filter

`injection.lua`, handing the request path to the response phase via dynamic metadata, body buffering.

Material: `envoy_scripts/README.md`

5.4 Envoy with a WebAssembly Filter

Rust proxy-wasm: configuration compiled in `on_configure`, fewer host calls, a byte-level splice. A proxy-wasm filter cannot answer locally from the body callback.

Material: `wasm-filter/README.md`, `wasm-filter/src/lib.rs`

5.5 Apache httpd with mod_lua

The shared `wadm.lua` core and its three hook scripts; `LuaScope` and `LuaCodeCache`.

Material: `apache_scripts/README.md`

5.6 Levelling the Edges

Product defaults levelled before any comparison: upstream keep-alive, access logs, code caching. Baselines are the RPDL configuration minus RPDL. The residual differences are listed in a table.

Material: `docs/EDGE_LEVELING.md`, `docs/THESIS_NOTES.md` (2026-09-17, 2026-09-18)

5.7 Verifying Equivalence

`parity_check.py` sends 13 requests to every edge and diffs the responses, the alert lines and the requests that reached the origin.

Material: `benchmarks/parity_check.py`

6 Evaluation Methodology

How the costs were measured, and why the measurements can be trusted.

6.1 Goals and Metrics

A table mapping each metric to its question. RQ1: latency added (median and p95). RQ2: CPU per request, capacity. RQ3: share of intact responses, alert recall, false positives. RQ4: in-edge timer cost per kind.

6.2 Testbed

The VM, the cpuset layout, guest tuning, image versions.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4

6.3 Workload

k6 iterations of nine requests: a priming `GET`, four `GET` probes and four SQLi `POST`s. Williams-balanced order [16]; arrival rates of 1, 10 and 100 iterations/s.

Material: `benchmarks/test.js`

6.4 Measurement Planes

In-edge timers (µs), end-to-end latency from k6 (ms), CPU per request from cgroup `cpu.stat`.

6.5 Run Validity

The `assess_run()` gates, the origin-CPU negative control, and detection of host freezes.

Material: `benchmarks/wadm_timings.py`, `docs/THESIS_NOTES.md` (2026-10-07)

7 Results

One section per research question, each ending with its answer.

7.1 RQ1: Latency Added

7.1.1 Median Latency per GET across Rates

+109 to +218 µs per `GET` (13–23%), resolved in every `GET` cell.

Material: `results/paired_headline.md`, `plots/paired_latency_added.png`

7.1.2 Tail Latency

p95 grows more than the median on every edge. Apache at 100 iterations/s collapses.

7.1.3 Request Position and Reconnects

The first request after the think-time sleep pays a wake-up cost. Apache closes the connection after the trap's 500.

Material: `docs/THESIS_NOTES.md` (2026-09-21, §5b), `plots/paired_latency_shift.png`

7.1.4 Answer to RQ1

One paragraph: the number per edge, how it changes with load, and how certain it is.

7.2 RQ2: CPU and Capacity

7.2.1 CPU per Request

CPU added per request: OpenResty +33, Envoy+Lua +41, WASM +123, Apache +200 µs.

7.2.2 Cost Attribution

The in-edge timers cover 5–22% of that CPU; the rest is VM entry and exit, body filters and log writes.

Material: `plots/cost_attribution.png`

7.2.3 The Cost of Measuring

The timing log lines alone cost about as much as RPDL itself on OpenResty and Envoy+Lua.

7.2.4 Per-Mechanism Cost and Page Size

The `GET` path against the trap; injection cost grows linearly with body size up to 768 KiB.

Material: `results/mechanisms.md`, `plots/mechanism_cpu.png`, `plots/size_sweep.png`

7.2.5 Answer to RQ2

CPU and capacity per edge, and where the cost arises.

7.3 RQ3: Functional Correctness

7.3.1 Response Integrity under Load

Share of intact responses per probe and rate. Apache is the only edge that falls below 100%.

Material: `results/paired_functional.md`

7.3.2 Detection Recall and False Positives

`WADM ALERT` and `WADM TRAP` lines counted against expected; zero on bare.

7.3.3 Answer to RQ3

What breaks, on which edge, and whether detection is complete.

7.4 RQ4: Influence of the Extension Runtime

7.4.1 Per-Kind Operation Cost

In-edge timer distributions per honeytoken kind and edge.

Material: `plots/paired_internal_timers.png`

7.4.2 The SQLi Trap: Scan Depth versus Decoding

In the encoding-by-outcome 2×2, decoding costs 0.5–2.5 µs on the Lua edges and nothing on WASM; scan depth is visible only on Apache's interpreted PUC Lua.

Material: `results/sqli.md`, `docs/THESIS_NOTES.md` (2026-09-21)

7.4.3 Stability of Apache mod_lua

Segfaults driven by cumulative requests in the body filter, and the cost of fixing them with `LuaScope conn`.

Material: `docs/BENCHMARK_METHODOLOGY.md` §4b–4c, `plots/apache_remediation.png`

7.4.4 Answer to RQ4

How much of RQ1–RQ3 is explained by the runtime rather than by the deception logic.

8 Discussion

What the results mean, and how far they can be trusted.

8.1 Synthesis across Research Questions

One table of edges against the four answers.

8.2 Choosing an Edge for Deception

A trade-off summary per edge: latency, CPU, capacity, robustness.

8.3 Where the Cost Comes From

Runtime entry and exit, body buffering and log writes, not the detection logic.

8.4 Deployment Implications

Compressed bodies, range requests, preserved ETags, body-size limits.

8.5 Threats to Validity

A single VM host, synthetic traffic, moving image tags, timer resolution.

8.6 Limitations

POST-body inspection exists on only two edges; no real attacker traffic; no TLS or HTTP/2.

9 Conclusion and Future Work

9.1 Summary

RQ1–RQ4 answered in one or two sentences each, then the overall contribution.

9.2 Future Work

POST-body inspection on all four reverse-proxy platforms, injecting into compressed bodies, a field study with real attackers, TLS and HTTP/2.

A Configuration Reference

Every `config.json` field per honeytoken kind, and the SQLi trap's fields.

Material: `CONTEXT.md`

B Parity-Check Request Set

The 13 requests and the documented exclusions.

C Reproducing the Benchmarks

Commands, machine configuration, run durations.

Material: `docs/BENCHMARK_METHODOLOGY.md` §3–4

D Additional Result Tables

Per-rate tables and figures.

Material: `benchmarks/results/plots/by_rate/`

Sources Used for This Proposal

[1] OWASP Foundation, *Web Security Testing Guide, Version 4.2*, 2020. Tests WSTG-INFO-03 and WSTG-INFO-05. Release of 03.12.2020: `https://github.com/OWASP/wstg/releases/tag/v4.2`. Accessed: 12.10.2026.

[2] X. Han, N. Kheir, and D. Balzarotti, "Evaluation of deception-based web attacks detection," in *Proceedings of the 2017 Workshop on Moving Target Defense (MTD)*, pp. 65–73, 2017.

[3] L. Spitzner, "Honeypots: Catching the insider threat," in *Proceedings of the 19th Annual Computer Security Applications Conference (ACSAC)*, pp. 170–179, 2003.

[4] X. Han, N. Kheir, and D. Balzarotti, "Deception techniques in computer security: A research perspective," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–36, 2018.

[5] M. Kahlhofer and S. Rass, "Application layer cyber deception without developer interaction," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 416–429, 2024.

[6] D. Fraunholz, D. Reti, S. D. Anton, and H. D. Schotten, "Cloxy: A context-aware deception-as-a-service reverse proxy for web services," in *Proceedings of the 5th ACM Workshop on Moving Target Defense (MTD)*, pp. 40–47, 2018.

[7] D. Reti, T. Angeli, and H. D. Schotten, "Honey infiltrator: Injecting honeytoken using netfilter," in *Proceedings of the IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 465–469, 2023.

[8] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz, D. Zhuo, and R. Mahajan, "Dissecting overheads of service mesh sidecars," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, pp. 142–157, 2023.

[9] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.

[10] L. Spitzner, "Honeytokens: The other honeypot." SecurityFocus, July 2003. `http://www.securityfocus.com/infocus/1713`.

[11] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, "Baiting inside attackers using decoy documents," in *Security and Privacy in Communication Networks (SecureComm)*, pp. 51–70, 2009.

[12] A. Juels and R. L. Rivest, "Honeywords: Making password-cracking detectable," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pp. 145–160, 2013.

[13] N. Virvilis, B. Vanautgaerden, and O. S. Serrano, "Changing the game: The art of deceiving sophisticated attackers," in *6th International Conference on Cyber Conflict (CyCon)*, pp. 87–97, 2014.

[14] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[15] G. Heiser, "Systems benchmarking crimes." `https://gernot-heiser.org/benchmarking-crimes.html`. Accessed: 07.10.2026.

[16] E. J. Williams, "Experimental designs balanced for the estimation of residual effects of treatments," *Australian Journal of Scientific Research, Series A*, vol. 2, no. 2, pp. 149–168, 1949.